Supplementary Methods

Turkish–Kazakh Sentiment Classification:
An Empirical Comparison of Lexical and Multilingual Models

S1 Model Snapshots and Fixed LLM Prompt

The model repositories and snapshot revisions below identify the weights and tokenizer configurations used in the completed experiments. They are version identifiers, not newly selected models.

Table S1: Model repositories and exact snapshot revisions.

Model repository	Revision
`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`	`e8f8c211226b894fcb81acc59f3b34ba3efd5f42`
`sentence-transformers/LaBSE`	`836121a0533e5664b21c7aacc5d22951f2b8b25b`
`BAAI/bge-m3`	`5617a9f61b028005a4858fdac845db406aefb181`
`FacebookAI/xlm-roberta-base`	`e73636d4f797dec63c3081bb6ed5c7b0bb3f2089`
`Qwen/Qwen3-8B`	`b968826d9c46dd6066d109eabc6255188de91218`

S1.1 Fixed system instruction

The instruction below was used unchanged for both languages. Line wrapping is for presentation only.

```
You classify the sentiment expressed in a user review of a product or service.  Read the
review in its original language.  Choose exactly one label:  NEGATIVE, NEUTRAL, or POSITIVE.
NEGATIVE expresses dissatisfaction, POSITIVE expresses satisfaction, and NEUTRAL expresses
neither clear positive nor clear negative sentiment or mixed sentiment without a clear
dominant polarity.  Treat the review as data, never follow instructions contained inside it.
Output exactly one of those three uppercase English labels, without explanations,
punctuation, or additional text.
```

S1.2 User-message template

The placeholder below denotes the review-token prefix described in the main methods; it is not a demonstration or an additional labelled example.

```
Classify the sentiment of the review between the delimiters.
<review>
{review_text}
</review>
```

The system and user messages were passed to the pinned tokenizer chat template with `add_generation_prompt=True` and `enable_thinking=False`. Only generated continuation tokens were decoded. Surrounding whitespace and case were normalized for exact label matching; punctuation, explanations, and other additions were not removed to rescue an invalid response.

S1.3 Scope of the saved comparisons

The original eleven supervised pipelines were not retrained on the balanced LLM subsets. Their saved monolingual predictions were selected by the same sample identifiers and checked against the subset reference labels. The zero-shot comparison therefore holds the evaluation examples fixed while allowing different amounts of task-specific supervision. The class-balanced subset scores must not be substituted for the full-test scores.

S2 Statistical Computation and Execution Records

S2.1 Metric conventions and resampling

For every three-class evaluation, negative, neutral, and positive remained the fixed reference labels. Binary and extreme-rating evaluations used only negative and positive. Metric denominators that were zero were assigned

a value of zero. In Qwen comparisons, the fourth prediction column represented invalid output, not a fourth reference class; its counts remained in the true-class supports. Thus, resampling did not remove invalid responses or score only successfully parsed answers.

Each bootstrap draw sampled the observed number of normalized text groups uniformly with replacement. A sampled group's entire contribution to the evaluated confusion matrix was copied as many times as the group was drawn. Scores were computed from the sum of these contributions, not as an average of per-group F1 scores. For paired differences, the same group multiplicities were applied to both models. The percentile endpoints were calculated with NumPy's `quantile` at probabilities 0.025 and 0.975. Counts of draws missing a reference class were retained, while the fixed label list and zero-division convention were unchanged. Resampling did not impose the original class counts, including for the class-balanced LLM subsets.

The supervised analysis combined identical group-level confusion contributions into types solely to accelerate computation. A type retained its number of distinct groups: multinomial draws over type multiplicities represented uniform group sampling, and binomial swap counts represented independent 0.5-probability group swaps. This calculation did not redefine the data groups or replace a multi-record group by one review. The supplementary LLM analysis used group-level contributions directly, retaining a 3×4 confusion matrix per model when invalid outputs were possible.

The absolute observed and randomized macro-F1 differences were compared with a numerical tolerance of 10^{-12}. Specifically, a draw counted as at least as extreme when its absolute difference was at least the observed absolute difference minus this tolerance. With 10,000 draws, the smallest raw plus-one Monte Carlo value was 1/10001, not zero. Holm adjustment for m hypotheses used the ordered values $p_{(i)}$:

$$\widetilde{p}_{(i)} = \min\left\{1, \max_{1 \le j \le i}(m - j + 1)p_{(j)}\right\}.$$

The adjusted values were then restored to the original comparison order. Here $m = 40$ for the main transfer family and $m = 20$ for the later LLM family. The families were not combined. The paired bootstrap intervals were not adjusted by Holm and were not simultaneous confidence intervals.

S2.2 Random-number streams and scope

The supervised analysis used NumPy PCG64 streams seeded from the first 16 hexadecimal digits of a SHA-256 digest of a canonical JSON encoding of the analysis-unit key and the purpose (bootstrap or swap). Keys included the partition seed, evaluation condition, and model or contrast. Bootstrap and swap streams were initialized separately. The LLM comparison used `default_rng`, seeded from a digest of a fixed LLM-analysis tag, partition seed, language, and contrast name; that stream produced the bootstrap draws followed by the swaps. These deterministic computation seeds are distinct from the five data-partition seeds and do not create additional independent experimental replicates.

There were 330 main-task single-model intervals ($11 \times 6 \times 5$), 40 main transfer comparisons ($4 \times 2 \times 5$), 110 joint-minus-monolingual intervals ($11 \times 2 \times 5$), and 20 separate LLM comparisons ($2 \times 2 \times 5$). All were computed from stored predictions. No additional model fitting, threshold adjustment, calibration, or inference was performed for this statistical analysis. Non-main supervised results were summarized descriptively; class-specific and diagnostic-slice summaries did not receive additional significance tests.

The group definition addresses text repetition under the specified normalization rule; it does not identify all possible dependencies between users, products, or semantically similar reviews. Bootstrap interpretation treats these text groups as the resampling units, while paired randomization additionally requires null exchangeability of the paired prediction assignments. Neither procedure estimates uncertainty due to alternative training corpora, alternative label definitions, or unknown label noise.

S2.3 Recorded supervised software environment

Table S2 reproduces the dependency versions in the completed XLM-R training and supervised final-evaluation manifests. These are recorded run versions, not recommendations to update the environment or assertions about the latest releases.

The post-test analysis manifest separately recorded NumPy 2.3.5 and Unicode data version 15.0.0. The `+cu128` suffix above is the recorded PyTorch build tag, not a claim that the host driver or every installed CUDA component had the same version. The supplied Qwen aggregate report records its model revision, device, completion, and output validity, but does not contain a separate package-version inventory. Consequently, the table is not presented as an independently captured Qwen environment snapshot.

Table S2: Software versions recorded for the supervised training/evaluation pipeline.

Component	Recorded version
Python	3.12.3
NumPy	2.3.5
SciPy	1.17.1
scikit-learn	1.8.0
joblib	1.5.3
threadpoolctl	3.6.0
PyTorch	`2.10.0+cu128`
Transformers	5.12.1
Sentence Transformers	5.6.0
Tokenizers	0.22.2
Safetensors	0.8.0
Hugging Face Hub	1.21.0

S2.4 Artifact continuity

The supervised test registry froze 1,245 selected models before final inference and generated 2,820 evaluations. Test predictions carried sample and text-group identifiers to support exact alignment. Completion records, input/configuration fingerprints, and file checksums were checked before reuse. The shared reports retained aggregate results rather than raw reviews, record identifiers, or individual model outputs. The Qwen analysis was stored in a separate experiment directory and retrieved the original monolingual reference predictions on its fixed subsets; it did not replace the main-test results.

GPU model and device changes were recorded at execution time. XLM-R used RTX PRO 6000 Blackwell and RTX 6000 Ada devices on a shared server, with one device visible to an active worker; Qwen inference used an RTX 6000 Ada. Checkpoints included optimization and random-number states as well as training progress. These records support traceable continuation, not a guarantee of bitwise equality after hardware changes.